Chapter 11 — Networks

Luca De Vecchis

Jan 2026

Contents

1 Flows **2**
1.1 Conceptual Positioning . . . 2
1.2 Flow Networks: Formal Definition . . . 2
1.3 Definition of a Flow . . . 2
1.4 Source and Sink Behavior . . . 3
1.5 Value of a Flow . . . 3
1.6 Structural Interpretation . . . 3
1.7 Feasible vs. Maximum Flows . . . 4
1.8 Comparison with Undirected Graphs . . . 4

2 Cuts **4**
2.1 Motivation and Duality Perspective . . . 4
2.2 Definition of an s–t Cut . . . 4
2.3 Cut-Set and Directionality . . . 5
2.4 Capacity of a Cut . . . 5
2.5 Fundamental Inequality: Flow vs. Cut . . . 5
2.6 Minimum Cuts . . . 6
2.7 Cuts vs. Vertex Cuts (Preview) . . . 6
2.8 Structural Insight . . . 6

3 The Max-Flow Min-Cut Theorem **6**
3.1 Statement of the Theorem . . . 7
3.2 What the Theorem Really Says . . . 7
3.3 Immediate Consequences . . . 7
3.4 Proof Strategy Overview . . . 7
3.5 Residual Network (Core Construction) . . . 8
3.6 Augmenting Paths . . . 8
3.7 Constructing a Cut from a Maximum Flow . . . 8
3.8 Why This Cut Is Minimum . . . 9
3.9 Logical Structure of the Proof . . . 9

1 Flows

1.1 Conceptual Positioning

Network flow theory sits at the intersection of graph theory, combinatorial optimization, and linear programming. While earlier chapters treated graphs primarily as static combinatorial objects, here we introduce *dynamics*: quantities that move through a graph subject to constraints.
Formally, we work with directed graphs endowed with capacities, and we study how much "stuff" (flow) can be transmitted from a designated *source* to a designated *sink*.
Typical interpretations include:

- commodities in transportation networks,
- data packets in communication networks,
- current in electrical networks,
- matchings and connectivity problems (via reductions).

1.2 Flow Networks: Formal Definition

A *flow network* is a quadruple

$$N = (D, s, t, c)$$

where:

- $D = (V, A)$ is a directed graph,
  - V is the vertex set,
  - $A \subseteq V \times V$ is the set of arcs (directed edges),
- $s \in V$ is the *source*,
- $t \in V$ is the *sink*, with $s \neq t$,
- $c : A \to \mathbb{R}_{\geq 0}$ is the *capacity function*.

Intuitively, for an arc $(u, v) \in A$, the value $c(u, v)$ represents the maximum amount of flow that can be sent directly from u to v.

1.3 Definition of a Flow

A *flow* in the network is a function

$$f : A \to \mathbb{R}_{\geq 0}$$

satisfying the following constraints.
pag 192 image

(i) Capacity Constraint For every arc $(u, v) \in A$,

$$0 \leq f(u, v) \leq c(u, v).$$

This is a local feasibility condition: no arc may carry more flow than its capacity.

(ii) Flow Conservation (Kirchhoff's Law) For every vertex $v \in V \setminus \{s, t\}$,

$$\sum_{(u,v)\in A} f(u, v) = \sum_{(v,w)\in A} f(v, w).$$

That is, the total inflow equals the total outflow at every intermediate vertex. Vertices other than s and t neither create nor destroy flow.

1.4 Source and Sink Behavior

The conservation condition is *not* imposed on the source s or the sink t.
Define:

$$\text{out}(v) = \sum_{(v,w)\in A} f(v, w), \qquad \text{in}(v) = \sum_{(u,v)\in A} f(u, v).$$

Then:

- At the source s: $\text{out}(s) \geq \text{in}(s)$,
- At the sink t: $\text{in}(t) \geq \text{out}(t)$.

The source emits flow, while the sink absorbs it.

1.5 Value of a Flow

The *value* of a flow f, denoted $|f|$, is defined as the net flow out of the source:

$$|f| = \sum_{(s,v)\in A} f(s, v) - \sum_{(v,s)\in A} f(v, s).$$

Equivalently, it can be expressed as the net flow into the sink:

$$|f| = \sum_{(u,t)\in A} f(u, t) - \sum_{(t,v)\in A} f(t, v).$$

These two expressions are equal as a direct consequence of flow conservation at all other vertices.

1.6 Structural Interpretation

From a linear-algebraic viewpoint:

- each arc corresponds to a variable $f(u, v)$,
- capacity constraints are box constraints,
- conservation constraints are linear equalities,
- the flow value is a linear objective function.

Thus, the maximum flow problem is a linear programming problem. However, its constraint matrix has special structure, enabling purely combinatorial algorithms.

1.7 Feasible vs. Maximum Flows

At this stage, it is important to distinguish:

- a *feasible flow*, which satisfies capacity and conservation constraints,
- a *maximum flow*, which is a feasible flow of maximum possible value.

This distinction will be crucial later, particularly in the Max-Flow Min-Cut Theorem and in feasible flow problems with demands (Section 11.5).

1.8 Comparison with Undirected Graphs

Although flow networks are directed, undirected graphs can be accommodated by replacing each undirected edge $\{u, v\}$ with two arcs:

$$(u, v) \quad \text{and} \quad (v, u),$$

each with capacity $c(\{u, v\})$.

This modeling choice is not innocent:

- it allows asymmetric usage,
- but forbids simultaneous bidirectional flow whose total exceeds the capacity.

pag 193 image

2 Cuts

This section introduces the dual object to flows. If flows represent movement through a network, cuts represent obstructions to that movement. The deep interaction between these two notions culminates in the *Max-Flow Min-Cut Theorem*.

2.1 Motivation and Duality Perspective

At an abstract level:

- **Flows** ask: How much can we send from s to t?
- **Cuts** ask: How little must we remove to separate s from t?

This is a classic primal–dual relationship, with flows acting as primal feasible solutions and cuts serving as dual feasible certificates of upper bounds.

2.2 Definition of an s–t Cut

Let $N = (D, s, t, c)$ be a flow network with vertex set V.
An s–t *cut* is a partition of the vertex set

$$V = S \cup T$$

such that:

- $s \in S$,
- $t \in T$,
- $S \cap T = \varnothing$.

We denote the cut by (S, T).
Intuitively, vertices in S are "reachable from the source," while vertices in T lie on the sink side.
picture pag 194

2.3 Cut-Set and Directionality

Because the network is directed, the direction of arcs is essential.
The *cut-set* of (S, T) is

$$\delta^+(S) = \{(u, v) \in A \mid u \in S,\ v \in T\}.$$

Only arcs directed from S to T contribute to the cut:

- arcs from T to S do *not* contribute,
- arcs entirely within S or within T are irrelevant.

This asymmetry is crucial and has no direct analogue in undirected cut theory.

2.4 Capacity of a Cut

The *capacity* of a cut (S, T) is defined as

$$c(S, T) = \sum_{(u,v) \in \delta^+(S)} c(u, v).$$

Interpretation: the maximum total flow that can cross from S to T in a single step.

2.5 Fundamental Inequality: Flow vs. Cut

Let f be any feasible flow, and let (S, T) be any s–t cut. Then

$$|f| \leq c(S, T).$$

Proof Sketch (Important). Consider the net flow leaving the set S:

$$\sum_{u \in S} \big(\mathrm{out}(u) - \mathrm{in}(u)\big).$$

By flow conservation, all arcs with both endpoints in S cancel. What remains are arcs crossing the cut:

- forward arcs $(u, v) \in \delta^+(S)$ contribute positively,
- backward arcs $(v, u) \in \delta^-(S)$ subtract flow.

Thus,

$$|f| = \sum_{(u,v) \in \delta^+(S)} f(u, v) - \sum_{(v,u) \in \delta^-(S)} f(v, u) \leq \sum_{(u,v) \in \delta^+(S)} c(u, v) = c(S, T).$$

This inequality is purely combinatorial and does not rely on the maximality of the flow f.

2.6 Minimum Cuts

An *s–t minimum cut* is a cut (S, T) of minimum capacity:

$$\lambda = \min_{(S,T)} c(S, T).$$

Key observations:

- there may be multiple minimum cuts,
- the minimum cut value is finite if all capacities are finite,
- every cut provides an upper bound on the value of any feasible flow.

2.7 Cuts vs. Vertex Cuts (Preview)

At this point, cuts are arc-based.
Later (Section 11.4), we will study:

- vertex cuts,
- local connectivity,
- reductions of vertex problems to edge problems via vertex splitting.

This section is intentionally restricted to arc cuts.

2.8 Structural Insight

Cuts are:

- discrete,
- global objects,
- insensitive to local rerouting of flow.

Flows are:

- continuous,
- local,
- sensitive to network topology.

The fact that these two objects coincide in optimal value is highly nontrivial and is precisely the content of the next theorem.

3 The Max-Flow Min-Cut Theorem

This theorem is the structural backbone of network theory. Everything before it prepares the language; everything after it exploits its consequences.
We present a precise statement, followed by conceptual interpretation, and then a proof emphasizing structural insight rather than algorithmic detail.

3.1 Statement of the Theorem

Let

$$N = (D, s, t, c)$$

be a flow network.
Then:

$$\max\{\, |f| \mid f \text{ is a feasible } s\text{–}t \text{ flow} \,\} = \min\{\, c(S,T) \mid (S,T) \text{ is an } s\text{–}t \text{ cut} \,\}.$$

Moreover, there exist:

- a maximum flow f^*,
- a minimum cut (S^*, T^*),

such that

$$|f^*| = c(S^*, T^*).$$

Thus, the equality of extrema is attained simultaneously.

3.2 What the Theorem Really Says

The theorem asserts a perfect duality:

- flows describe what can be achieved,
- cuts describe what cannot be exceeded.

In networks, every obstruction is tight.
Equivalently:

> Every bottleneck in the network can be exactly saturated by some flow.

This is an exceptionally strong structural statement.

3.3 Immediate Consequences

Three fundamental corollaries follow immediately:

1. **Optimality certificates:** a cut certifies the optimality of a flow.
2. **Existence of maxima:** the supremum of feasible flow values is always attained.
3. **Algorithmic termination:** any algorithm that strictly increases flow must terminate.

3.4 Proof Strategy Overview

Several proofs of the theorem exist. At a doctoral level, the most revealing approach uses *residual networks* and *augmenting paths*, as it exposes the combinatorial mechanism underlying the min–max equality.
The core equivalence is:

$$\text{No augmenting path exists} \iff \text{A minimum cut is identified.}$$

3.5 Residual Network (Core Construction)

Let f be a feasible flow in N. The *residual network* D_f is defined as follows:

- the vertex set is V,
- for each arc $(u, v) \in A$:
  - the forward residual capacity is
  $$c_f(u, v) = c(u, v) - f(u, v),$$
  - the backward residual capacity is
  $$c_f(v, u) = f(u, v).$$

An arc appears in D_f if and only if its residual capacity is positive.
Interpretation: residual arcs encode all allowable local modifications of the current flow.
Picture page 205

3.6 Augmenting Paths

An *augmenting path* is a directed path from s to t in the residual network D_f.
If such a path exists:

- additional flow can be pushed along the path,
- the current flow cannot be maximum.

Hence:
$$f \text{ is a maximum flow} \quad \Longleftrightarrow \quad \text{no augmenting path exists in } D_f.$$

This equivalence is itself a nontrivial result.

3.7 Constructing a Cut from a Maximum Flow

Assume that f admits no augmenting path.
Define:
$$S = \{\, v \in V \mid v \text{ is reachable from } s \text{ in } D_f \,\}, \qquad T = V \setminus S.$$

Then:

- $s \in S$,
- $t \in T$ (otherwise an augmenting path would exist),

so (S, T) is an s–t cut.

3.8 Why This Cut Is Minimum

Consider arcs crossing the cut (S, T):

- If $(u, v) \in S \to T$, then
$$c_f(u, v) = 0 \quad \Rightarrow \quad f(u, v) = c(u, v),$$
so the arc is saturated.

- If $(v, u) \in T \to S$, then
$$f(v, u) = 0.$$

Therefore,
$$|f| = \sum_{(u,v) \in S \to T} f(u, v) = \sum_{(u,v) \in S \to T} c(u, v) = c(S, T).$$

By the inequality established in Section 11.2, this cut must be minimum.

3.9 Logical Structure of the Proof

The proof establishes:

- an *upper bound*: flow value $\leq$ cut capacity (always),
- *tightness*: when no augmentation is possible, equality holds.

This is the essence of the min–max principle.